Source → filter() → map() → (optional sorted/distinct) → Terminal Operation		
Terminal Operation	**Use Case**	**One-Line Code Example**
collect()	Accumulate elements	list.stream().collect(Collectors.toList());
collect()	Transform & group	users.stream().collect(Collectors.groupingBy(User::getDept));
collect()	Map → Reduce → Store	orders.stream().collect(Collectors.groupingBy(Order::getType, Collectors.summingDouble(Order::getAmount)));
-		
forEach()	Logging	list.stream().forEach(x -> logger.info(x.toString()));
forEach()	Printing	list.stream().forEach(System.out::println);
forEach()	Updating external systems	users.stream().forEach(u -> auditService.update(u));
reduce()	Combine values	list.stream().reduce(0, Integer::sum);
reduce()	Aggregate (sum / max / concat)	list.stream().reduce(Integer::max).orElse(0);

-		
count()	No custom logic allowed	list.stream().count();
-		
findFirst()	Conditional check before first match	list.stream().filter(x -> x > 10).findFirst();
-		
allMatch()	Validation logic	users.stream().allMatch(u -> u.getAge() >= 18);
allMatch()	Rule checks	orders.stream().allMatch(o -> o.isPaid());
-		
anyMatch()	Existence checks	users.stream().anyMatch(u -> u.getRole().equals("ADMIN"));
noneMatch()	Negative validation	users.stream().noneMatch(u -> u.isBlocked());